Q22: LLM vs. Agent — and Where Claude Code Sits

Paradise (nuc13)

October 2026

The question

- "What separates an LLM from an agent? Is Claude Code an agent or a harness?"
- Why it matters: most "agent" claims are really claims about software around a model
- Answer preview: **the loop, not the weights, supplies the agency; the harness owns the loop**

The agent spectrum

1. Single model call: prompt in, tokens out; caller owns everything
2. Tool-augmented call: model emits tool requests, runtime executes them (Toolformer)
3. The agentic loop: output fed back with tool results until a stop condition (ReAct)
4. Autonomous goal pursuit: goals, cross-episode memory, self-critique (Reflexion; surveys)

Anthropic: workflows (predefined code paths) vs. agents (model directs its own process)

The minimal agent: five parts

- **Model**: maps context $\rightarrow$ next action (an LLM — or 30 lines of rules)
- **Loop**: feed results back until a stop condition
- **Tools**: actions with effects; observations back into context
- **Memory**: transcript, scratchpads, retrieval
- **Stopping criterion**: finish action, budget, error detection

Drop the loop: a function call. Drop the stopping rule: `while True`.

The harness: who owns the control flow

The scaffold around the model:

- Prompt assembly, tool dispatch, result appending
- Context/memory management, retries, malformed-output handling
- Permissions, sandboxing, budgets, stop conditions

The model makes *local* decisions (one step, from current context);
the harness owns *global* behavior. They fail differently, improve independently, and are swappable.

Evidence that the harness matters

SWE-agent (Yang et al. 2024), same base LM:

- Non-interactive retrieval: 3.8% of SWE-bench resolved
- Agent + raw Linux shell: 11.0% (Lite split)
- Agent + designed agent–computer interface: 18.0% (Lite), 12.47% (full)

All of the gain is in the interface layer: guardrails, feedback, context management. Weights untouched $\Rightarrow$ agency is not only in the weights.

So: is Claude Code an agent?

- Claude Code = a **harness**: terminal scaffold managing tools, permissions, context, retries, headless mode
- Claude Code + model, running a task = an **agent** (model directs its own process)
- Both true — different layers. Best phrase: **agentic harness**
- Same analysis for Codex CLI and Hermes Agent

Caveats: the boundary moves (models absorb harness work over time);
"autonomy" = loop + memory + stopping rule, not a hidden optimizer

What the loop costs you

- **Non-stationary context**: the agent conditions on a context it is itself generating
- **Silent compounding failure**: misread filename $\rightarrow$ wrong edit $\rightarrow$ failing test $\rightarrow$ noisy traceback
- **Context rot**: retrieval degrades for information mid-context (Lost in the Middle); harnesses compact/summarize
- **Error compounding**: per-step reliability p, k dependent steps $\Rightarrow$ success $\lesssim p^k$ (95% over 40 steps $\approx$ 13%)

Evaluation: why Q24 exists

- Trajectory, not response, is the unit of evaluation
- Model and harness must be scored separately — leaderboard rows conflate them
- Variance, cost, and overfitting are first-class (AI Agents That Matter)

The demo: same rule-based "model" — with and without the loop.
The loop supplies the agent-ness; the model is unchanged.

Reading

- Yao et al. 2023, ReAct (ICLR) — the loop pattern
- Shinn et al. 2023, Reflexion (NeurIPS) — self-critique between episodes
- Wang et al. 2024, survey of LLM-based autonomous agents — the module taxonomy
- Yang et al. 2024, SWE-agent (NeurIPS) — harness design moves the number
- Anthropic 2024–25, Building Effective Agents; Claude Code best practices